

Homework 6

Gabriela Rubio

April 9, 2026

1. (a) i. **Algorithm Description:** We can solve this algorithm by running a BFS from s on the graph G and following the edges in their given directions. We keep a visited array to track which vertices we reach and, after the BFS finishes, we check if every vertex was visited. If any one vertex wasn't visited by the time the BFS is done, the s can't reach it and we just return false. Otherwise, we return true.

ii. **Pseudocode:**

Algorithm CanSReachAll(G, s)

Input: Directed graph G with adjacency lists, source vertex s

Output: True if s can reach every vertex, false otherwise

```
for each vertex  $v$  in  $G$ :  
    visited[ $v$ ] = false
```

```
visited[ $s$ ] = true
```

```
 $Q$  = new Queue
```

```
 $Q$ .enqueue( $s$ )
```

```
while  $Q$  is not empty:
```

```
     $u$  =  $Q$ .dequeue()
```

```
    for each  $v$  in adjacencyList[ $u$ ]
```

```
        if visited[ $v$ ] == false:
```

```
            visited[ $v$ ] = true
```

```
             $Q$ .enqueue( $v$ )
```

```
for each vertex in  $G$ :
```

```
    if visited[ $v$ ] == false:
```

```
        return false
```

```
return true
```

- iii. **Correctness:** The algorithm is correct because the BFS from s visits all of the vertices reachable from s from directed edges. If $\text{visited}[v] = \text{true}$ after the BFS runs, there is a directed path from s to v . If $\text{visited}[v] = \text{false}$, then there's not a path. So checking that all of the vertices were visited will answer whether s can reach every vertex in G .
- iv. **Time Analysis:** Initializing the visited array takes $O(n)$, the BFS visits each vertex at most once and scans each edge at most once, so it takes $O(m + n)$, The final check that goes over all the vertices takes $O(n)$, so the total time taken is $O(m + n)$.
- (b) i. **Algorithm Description:** We can solve this algorithm by reversing the graph and noting that v can reach s in G if s can reach v in the reversed graph. So we first build the reversed graph of G by flipping every edge, then run one BFS from s on the reversed graph of G , and then check whether all vertices were visited.
- ii. **Pseudocode:**
- Algorithm CanAllReachS(G, s)
- Input: Directed graph G with adjacency lists, source vertex s
- Output: True if every vertex can reach s , false otherwise

```

for each vertex v in G:
    adjacencyListReversed[v] = {}

for each vertex u in G:
    for each neighbor v in adjacencyList[u]:
        adjacencyListReversed[v].append(u)

for each vertex v in G:
    visited[v] = false

visited[s] = true
Q = new Queue
Q.enqueue(s)

while Q is not empty:
    u = Q.dequeue()
    for each neighbor v in adjacencyListReverse[u]:
        if visited[v] == false:
            visited[v] = true
            Q.enqueue(v)

for each vertex v in G:
    if visited[v] == false:
        return false

return true

```

- iii. **Correctness:** The algorithm is correct because of the observation: there is a path $v \rightarrow s$ in G if there is a path $s \rightarrow v$ in G_{rev} . This is true because reversing all of the edges on a path $v \rightarrow \dots \rightarrow s$ will give a path $s \rightarrow \dots \rightarrow v$, and viceversa. So the BFS from s to the reversed graph will visit all of the vertices that can reach s in the original graph, and then checking that all vertices were visited correctly will answer the question.
 - iv. **Time Analysis:** Building the reversed graph scans every adjacency list and adds each edge once, which takes $O(m + n)$. The BFS on the reversed graph takes $O(m + n)$. The check takes $O(n)$. The total time complexity is $O(m + n)$.
2. (a) **Algorithm Description:** We can solve this algorithm by modifying a BFS to also count the number of shortest paths. We begin by keeping two arrays, one to keep the shortest distance from s to v , and another one to count the number of shortest paths from s to v . When we explore an edge (u, v) if v hasn't been seen yet, we set its distance and inherit the count u . If v has already been seen at the distance of $u + 1$, we found another shortest path, so we add the count of u to the count of v . Otherwise, the edge doesn't lead to a shortest path and we skip it. The final answer is the count of t .
- (b) **Pseudocode:**

Algorithm CountShortestPaths(G, s, t)

Input: Undirected graph G with adjacency lists, vertices s and t

Output: Number of shortest paths from s to t

```

for each vertex v in G:
    distance[v] = infinity
    count[v] = 0

```

```

distance[s] = 0
count[s] = 1
Q = new Queue
Q.enqueue(s)

```

```

while Q is not empty:

```

```

    u = Q.dequeue()
    for each neighbour v in adjacencyList[u]:
        if distance[v] == infinity:
            distance[v] = distance[u] + 1
            count[v] = count[u]
            Q.enqueue(v)

        else if distance[v] == distance[u] + 1:
            count[v] = count[v] + count[u]

return count[t]

```

- (c) **Correctness:** The algorithm is correct because the BFS explores vertices in a non-decreasing order of distance, so the invariant is that after processing all of the vertices at a distance d , then $\text{count}[v]$ holds the number of shortest paths from s to every vertex v at a distance that is less or equal to d . The base case is when $\text{count}[s] = 1$, because there is exactly one path of length 0 from s to itself. For any vertex v at a distance $d + 1$, every shortest path to v arrives through some neighbor u at a distance d through one single edge, so $\text{count}[v] = \sum \text{count}[u]$ over all u , which is what we compute. The goal is to get $\text{count}[t]$, which is the number of shortest paths from s to t .
- (d) **Time Analysis:** Initialization takes $O(n)$, the BFS visits each vertex at most once and also scans each edge at most once so it only takes $O(1)$ per edge. The total time is $O(m + n)$.
3. (a) **Algorithm Description:** We can solve this algorithm by using topological sort and dynamic programming. We can define an array $\text{count}[v]$ that holds the number of paths from s to v in G . The base case is $\text{count}[s] = 1$ and $\text{count}[v] = 0$ for all other vertices. We then process the vertices in topological order, and for each edge $(u \rightarrow v)$ we add $\text{count}[u]$ to $\text{count}[v]$. Since the graph is a DAG, when we process u all of the paths into u are already finalized, and the final answer is $\text{count}[t]$.

(b) **Pseudocode:**

```

Algorithm: CountAllPaths(G, s, t)
Input: DAG G with adjacency lists, vertices s and t
Output: number of paths from s to t

Run DFS on G
order[] = the vertices sorted by decreasing finish time

for each vertex v to G:
    count[v] = 0

count[s] = 1

for each vertex u in order[]:
    for each v in adjacencyList[u]:
        count[v] = count[v] + count[u]

return count[t]

```

- (c) **Correctness:** The algorithm is correct because it follows the steps of designing a dynamic programming algorithm. The subproblem is finding $\text{count}[v]$, the number of paths from s to v . The base case is $\text{count}[s] = 1$, since there is exactly one path from s to itself, and $\text{count}[v] = 0$ for all other vertices because no paths have been counted yet. For the dependency relation every path from s to v comes through some predecessor u with edge $(u \rightarrow v)$. Since we process vertices in topological order $\text{count}[u]$ is already correct when we process u . We then add $\text{count}[u]$ to $\text{count}[v]$ for each of those edges, which sums up all paths. If v is not reachable from s , $\text{count}[v]$ stays 0. The goal is $\text{count}[t]$, which is the number of paths from s to t .

- (d) **Time Analysis:** Topological sort through DFS takes $O(m + n)$, the DP loop visits each vertex once and each edge once, so it also takes $O(m + n)$. The total time is $O(m + n)$.
4. (a) **Algorithm Description:** We can solve this algorithm by using dynamic programming. We can define an 2D array $f[i][v]$ the holds the minimum weight of any path from s to v using at most i edges. The base case is $f[0][s] = 0$ and $f[0][v] = \infty$ for all $v \neq s$ because with zero edges we can only be at s with cost 0. For each i from 1 to k , we first let $f[i][v] = f[i-1][v]$ (inherit the best path using fewer edges), then for each edge $(u \rightarrow v)$, we update $f[i][v] = \min(f[i][v], f[i-1][u] + w(u, v))$. We process vertices in topological order. The final answer is $f[k][t]$.

(b) **Pseudocode:**

```

Algorithm: KLinkShortestPath(G, s, t, k)
Input: DAG G with adjacency lists and edge weights, vertices s and t,
       integer k
Output: Minimum weight of a path from s to t using at most k edges

order[] = topological sort of G

for each vertex v in G:
    f[0][v] = infinity
f[0][s] = 0

for i = 1 to k:
    for each vertex v in G:
        f[i][v] = f[i-1][v]

        for each vertex u in order[]:
            for each v in adjacencyList[u]:
                if f[i-1][u] + w(u,v) < f[i][v]:
                    f[i][v] = f[i-1][u] + w(u,v)

if f[k][t] == infinity:
    return "no k-link path exists"
return f[k][t]

```

- (c) **Correctness:** The algorithm is correct because it follows the steps of designing a dynamic programming algorithm. The subproblem is finding $f[i][v]$, which is the minimum weight path from s to v using at most i edges. The base case is $f[0][s] = 0$ and $f[0][v] = \infty$ for $v \neq s$, since with zero edges only s is reachable with a cost 0. For the dependency relation $f[i][v]$ depends only on row $i-1$ which is already computed. For each vertex v , either the best i -link path uses fewer than i edges or it uses exactly i edges and arrives through some last edge $(u \rightarrow v)$, giving cost $f[i-1][u] + w(u, v)$. We take the minimum over both cases. The goal is $f[k][t]$, the minimum weight of a path from s to t using at most k edges.
- (d) **Time Analysis:** Topological sort takes $O(m + n)$. The outer loop runs k times, and each iteration scans all n vertices and all m edges, so it takes $O(m + n)$ per iteration. The total time is $O(k(m + n))$. The space used by the DP table is $O(kn)$, but this can be reduced to $O(n)$ by keeping only the previous and current rows.